

Nível de Microarquitetura



UNIFEI

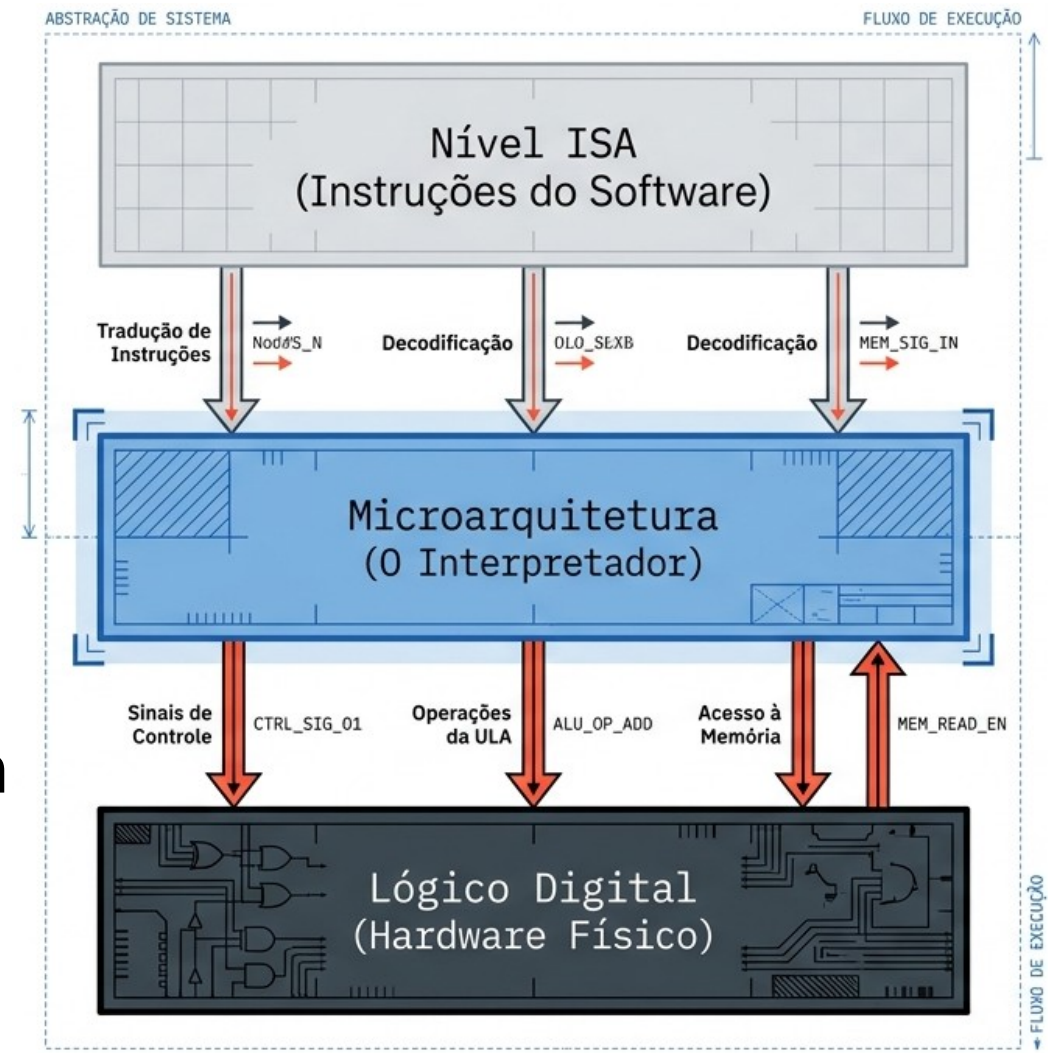
Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

O Nível da Microarquitetura

- A função deste nível é implementar o nível superior (Nível ISA).
- O projeto depende da meta: custo, desempenho ou simplicidade.
- Estudaremos a arquitetura **Mic-1**.



ISA vs Microarquitetura

- A Microarquitetura é a implementação física dos conceitos lógicos da ISA (Instruction Set Architecture).
- Máquinas RISCs implementam instruções simples que são executadas apenas com 1 ciclo de clock. E ISAs mais complexas como a do Core i7 que necessita de vários ciclos de clock para executar apenas 1 instrução.

Um exemplo de Microarquitetura

- Não existem princípios gerais para um projeto de microarquitetura, cada microarquitetura é um caso especial.
- Para a ISA de exemplo iremos utilizar um subconjunto da Java Virtual Machine que opera somente com números inteiros e iremos denominar IJVM.
- A microarquitetura conterá um micro programa (em ROM) cuja tarefa é buscar, decodificar e executar instruções IJVM.

Um exemplo de Microarquitetura

- Um modelo conveniente para o projeto de microarquitetura é pensar no projeto como um problema de programação no qual cada instrução no nível ISA é uma função a ser chamada por um programa mestre. O programa mestre é um laço simples, sem fim.
- O micro programa tem um conjunto de variáveis denominado estado do computador, que pode ser acessado por todas as funções.
- Instruções IJVM são curtas e fáceis. Cada instrução tem alguns campos, usualmente um ou dois, e cada um deles tem algumas finalidades específicas.

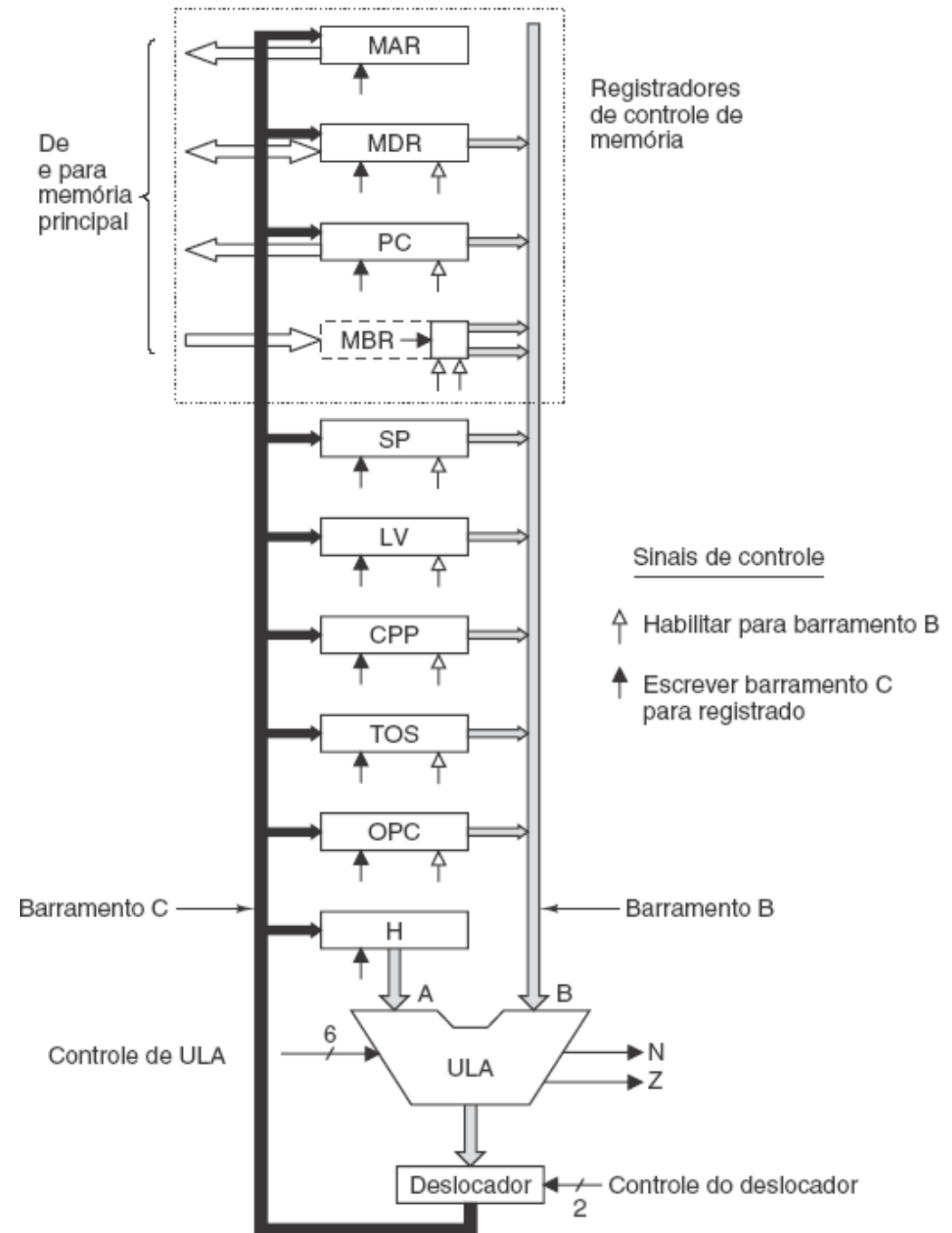
Um exemplo de Microarquitetura

- O primeiro campo de toda instrução é o **opcode** (abreviatura de **operation code** - código de operação), que indica a instrução informada, por exemplo, ADD ou BRANCH.
- Muitas instruções possuem campos adicionais que são operandos.
- Esse modelo de execução, às vezes é denominado ciclo **buscar-executar**, é útil um termos abstratos.

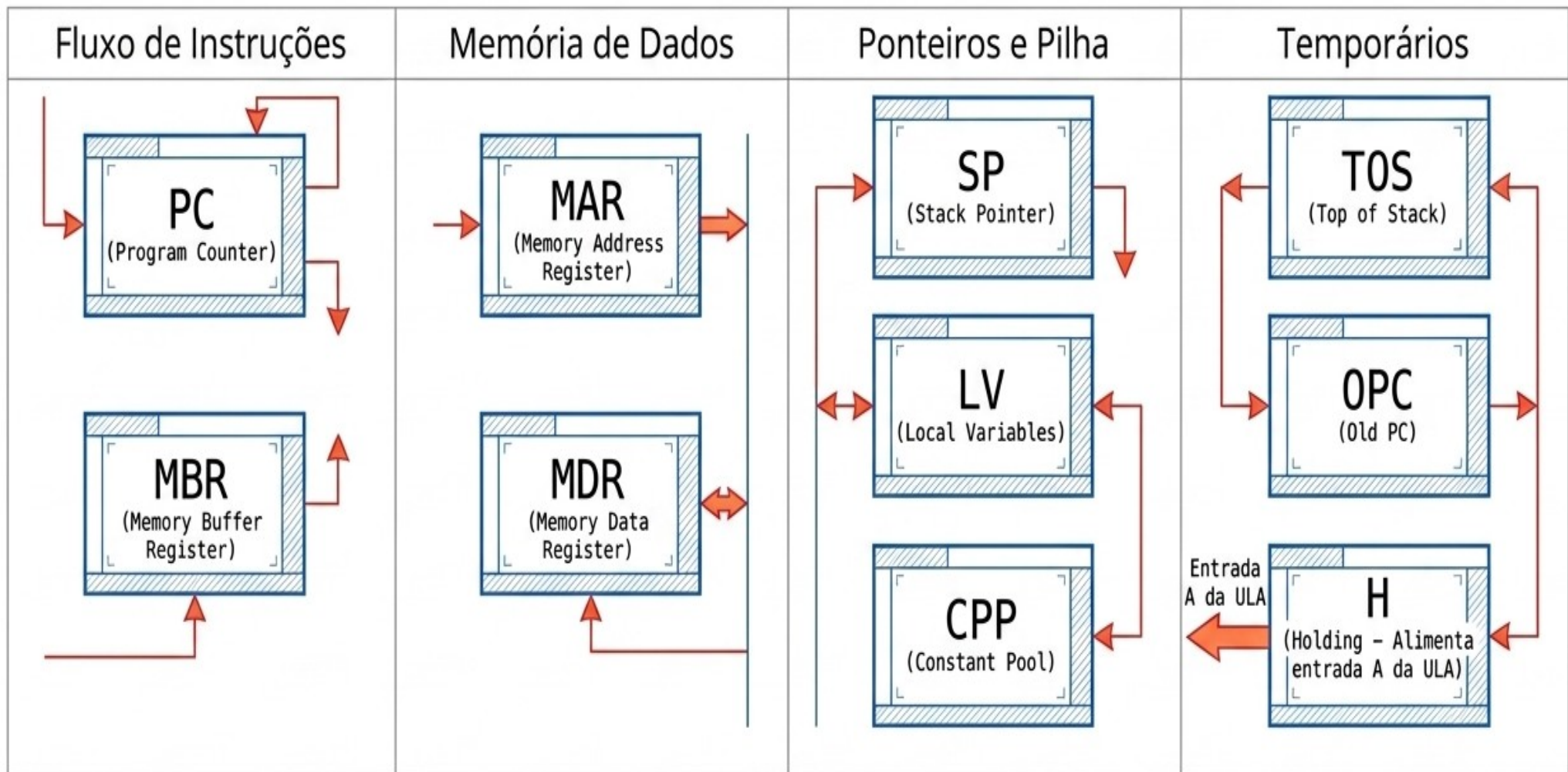
Caminho de dados

Caminho de dados é a parte da CPU que contém a ALU, suas entradas e saídas

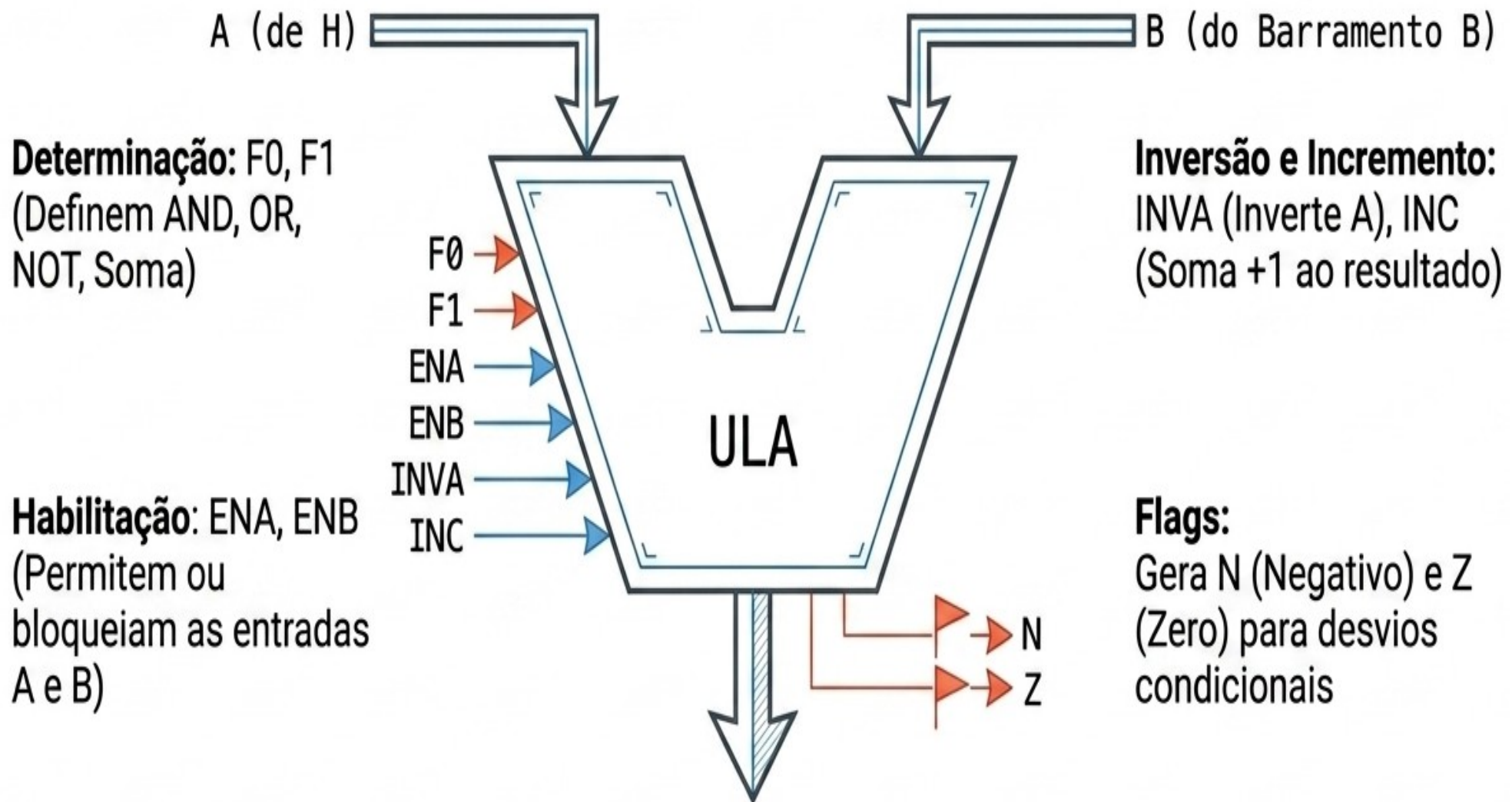
- Mic-1 é uma arquitetura microprogramada.
- Projetada para executar um subconjunto da JVM (IJVM).
- Controle é feito por microcódigo em ROM.

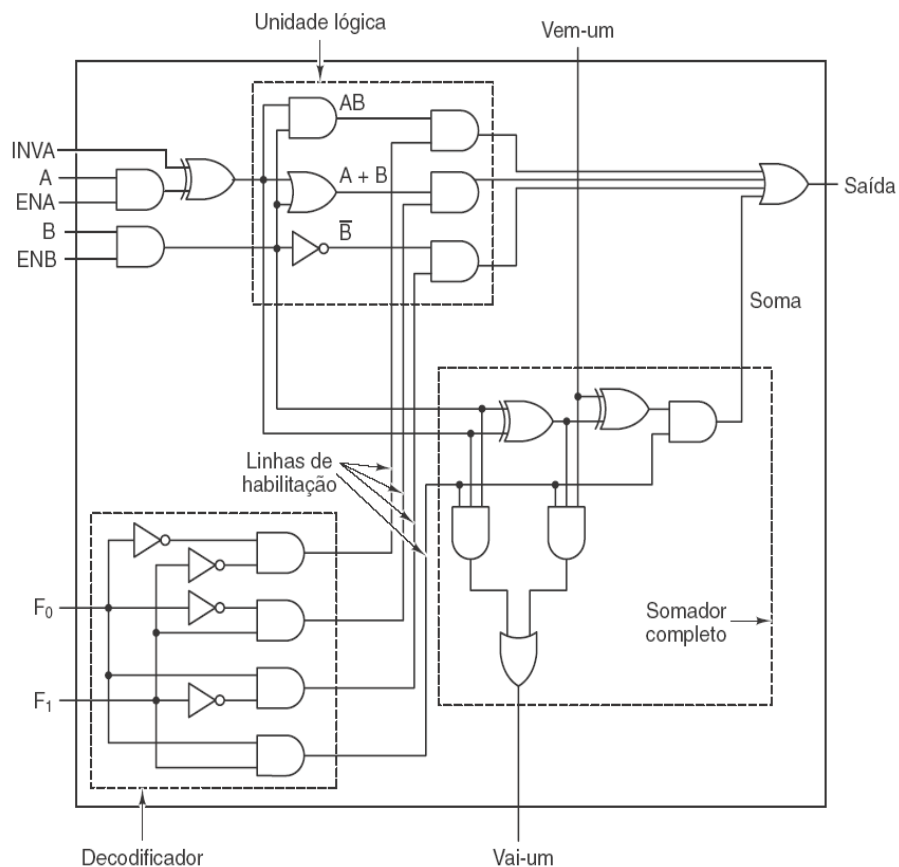


Arsenal de Registradores (32 bits)



O Motor: Unidade Lógica e Aritmética (ULA)



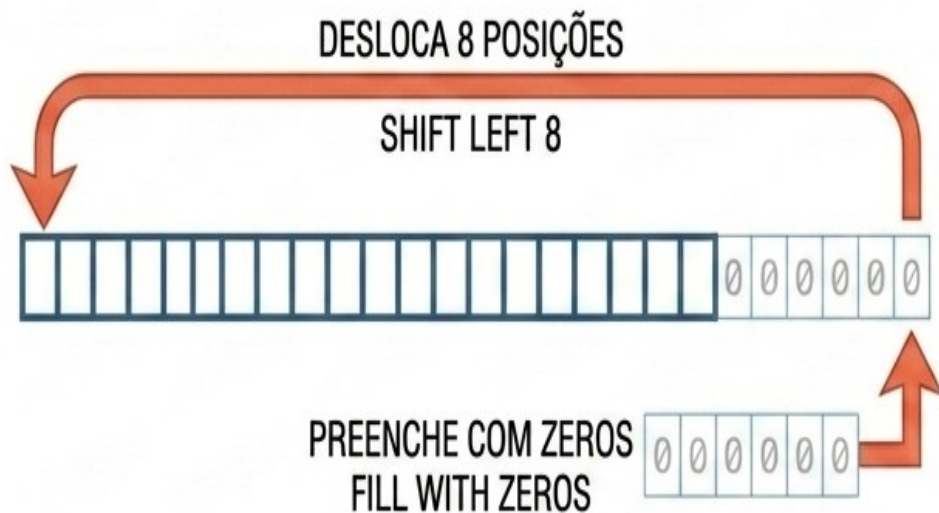


F_0	F_1	ENA	ENB	INVA	INC	Função
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	$A + B$
1	1	1	1	0	1	$A + B + 1$
1	1	1	0	0	1	$A + 1$
1	1	0	1	0	1	$B + 1$
1	1	1	1	1	1	$B - A$
1	1	0	1	1	0	$B - 1$
1	1	1	0	1	1	$-A$
0	0	1	1	0	0	$A \text{ AND } B$
0	1	1	1	0	0	$A \text{ OR } B$
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	-1

Combinações úteis de sinais da ALU e funções executadas.

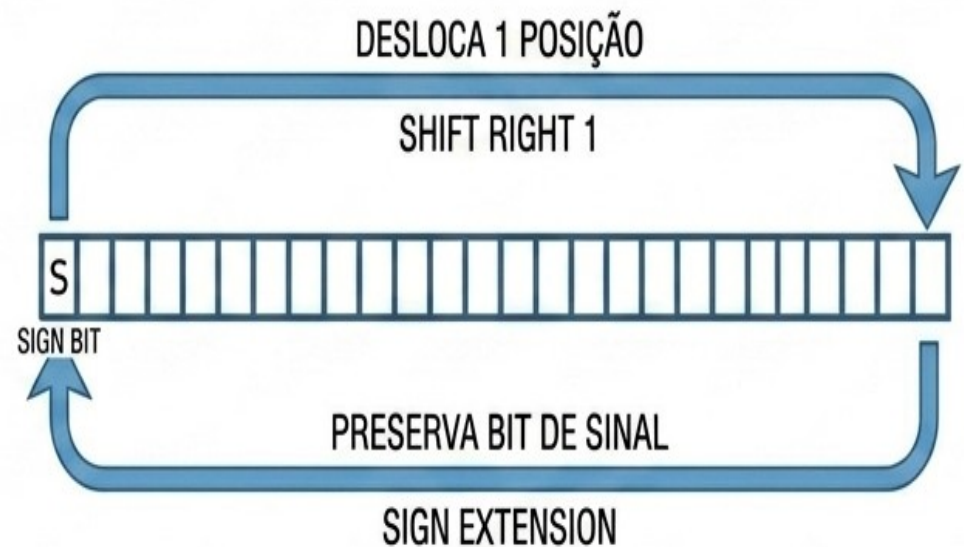
O Manipulador de Bits: Deslocador (Shifter)

SLL8 (Shift Left Logical)



Desloca o conteúdo 1 byte (8 bits) à esquerda, preenchendo os espaços com zeros.

SRA1 (Shift Right Arithmetic)



Desloca o conteúdo 1 bit à direita, preservando o bit de sinal (essencial para números negativos).

Temporização do caminho de dados

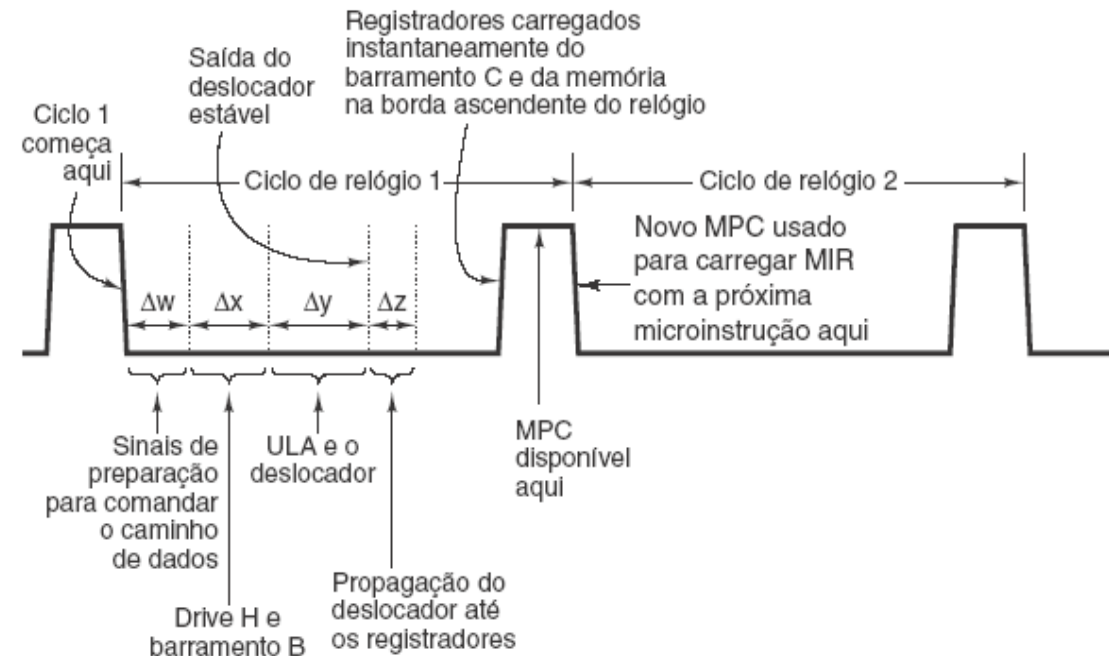
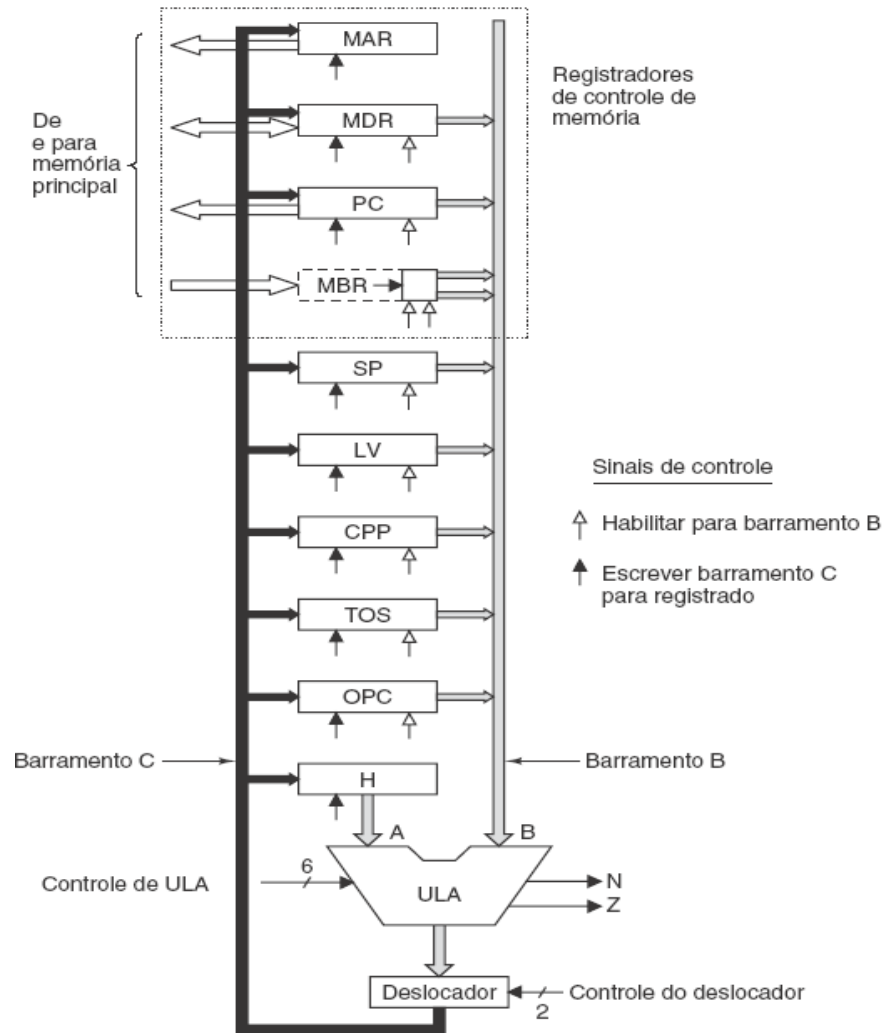
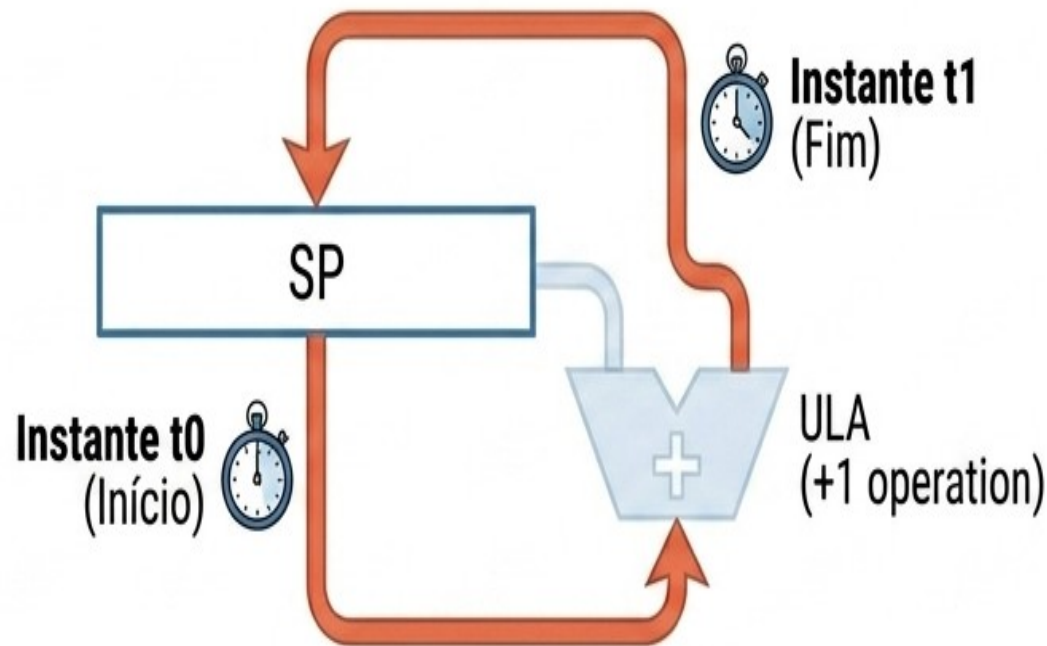


Diagrama de temporização de um ciclo do caminho de dados.

O Paradoxo da Leitura e Escrita

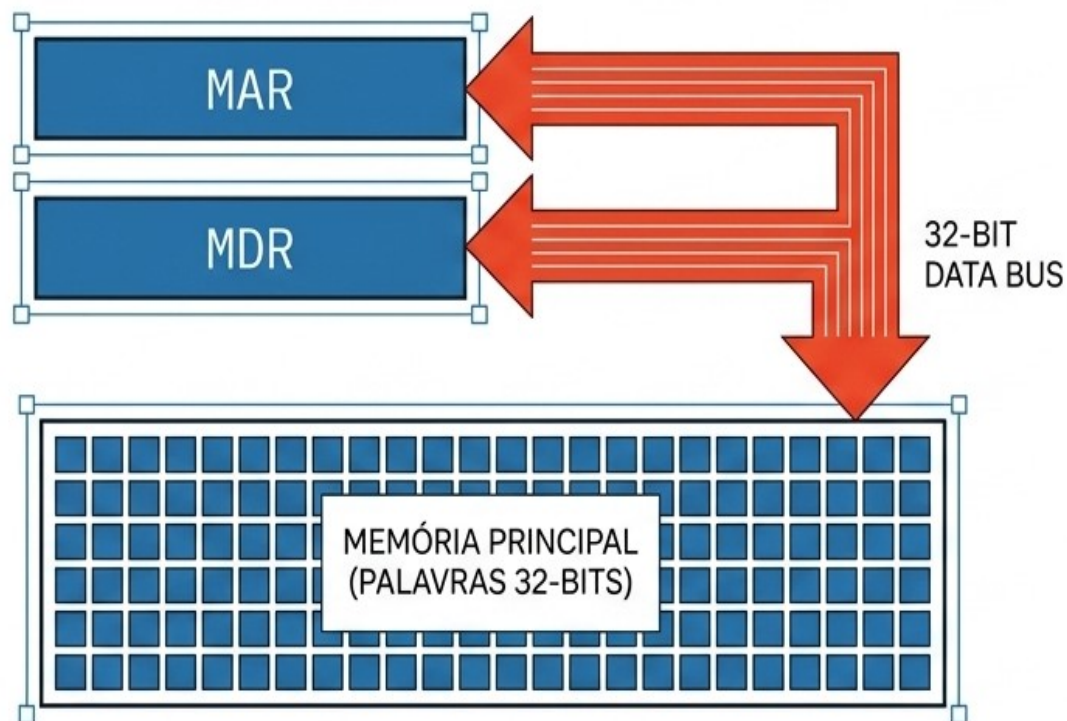
A Dúvida: Como ler o SP, incrementá-lo e salvar nele mesmo em um ciclo sem gerar lixo?



O Segredo Físico: A leitura e a escrita ocorrem em instantes físicos diferentes. A leitura é acionada no início (borda descendente) e o valor viaja pelo circuito. A escrita só é registrada no pulso final (borda ascendente), quando o sinal elétrico já está perfeitamente estável.

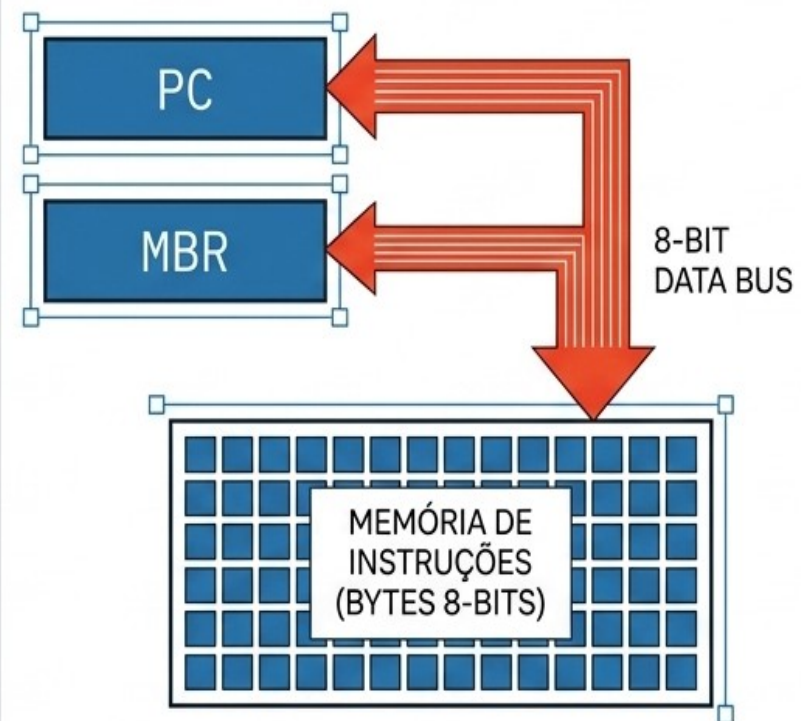
Conversando com a Memória Principal

Porta de Dados (32-bits)



- **MAR:** Aponta o endereço (em palavras). Não pode receber dados do barramento C.
- **MDR:** Lê e escreve dados na memória bidirecionalmente.

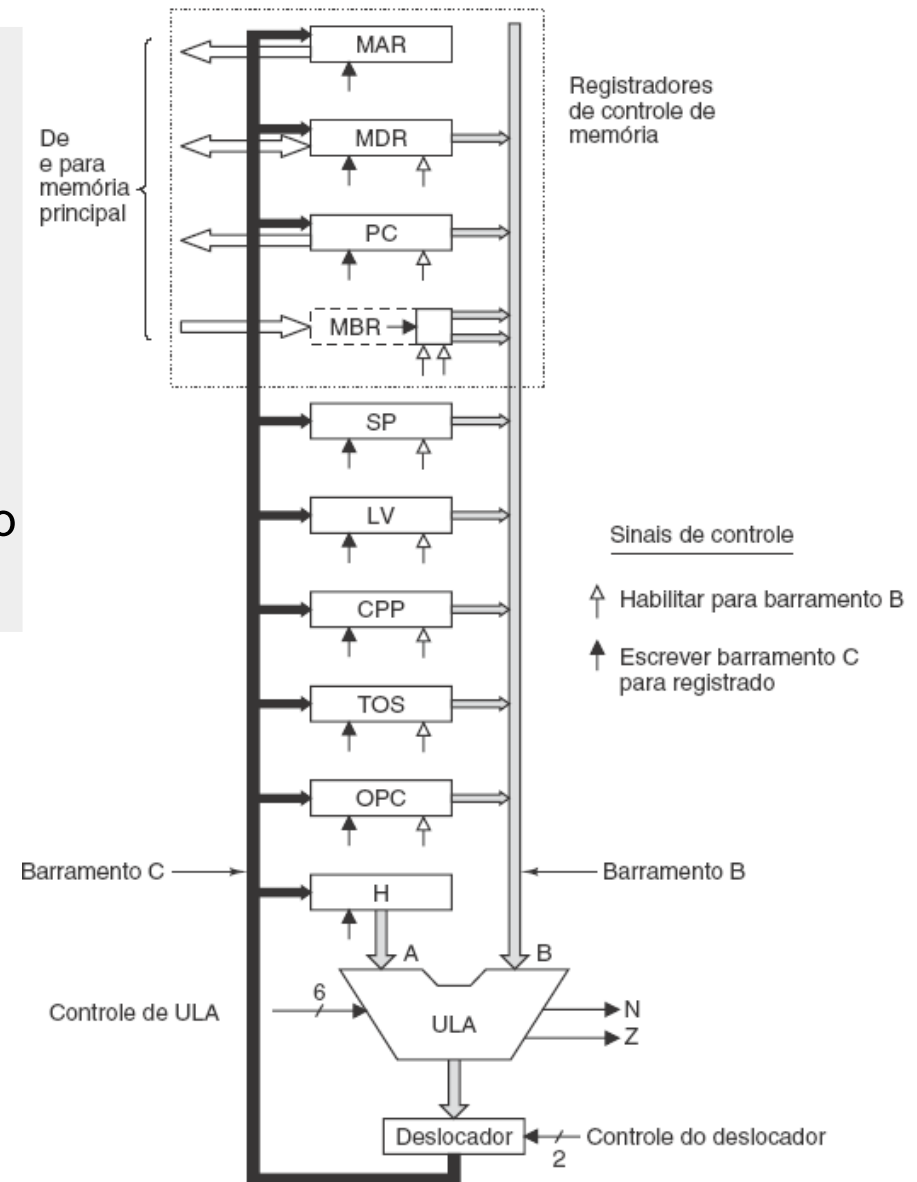
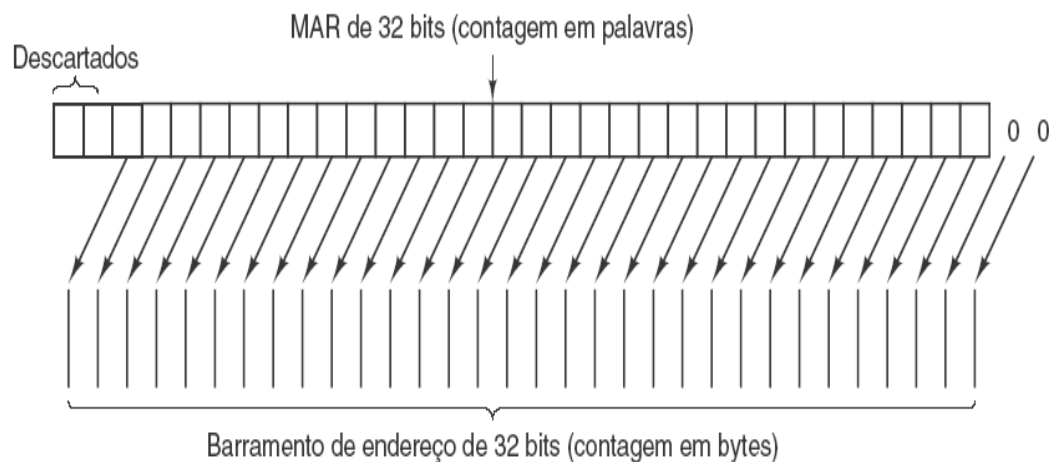
Porta de Instruções (8-bits)



- **PC:** Contador de programa, aponta para bytes.
- **MBR:** Buffer que busca (fetch) apenas a próxima instrução do programa.

O Truque de Alinhamento do MAR

- **O Problema:** A memória conta em bytes, mas o MAR endereça blocos inteiro de 32 bits (palavras)
- **A Solução Física:** Os engenheiros deslocaram a fiação. O pino 0 do MAR é ligado à linha 2 do barramento.
- **O Resultado:** O endereço é multiplicado por 4 puramente pela disposição dos fios no silício. Zero atraso, zero processamento extra.

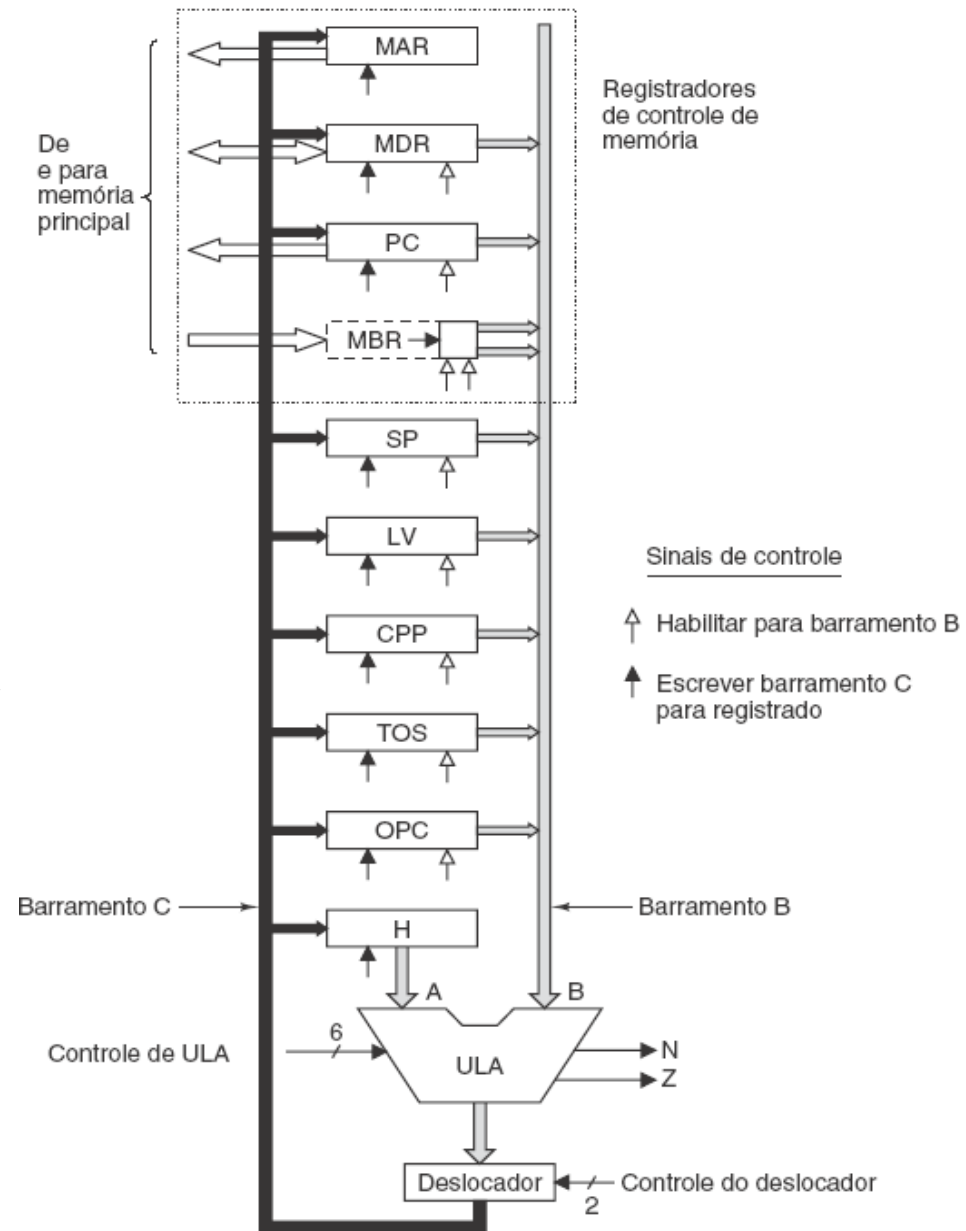


Mapeamento dos bits em MAR para o barramento de endereço.

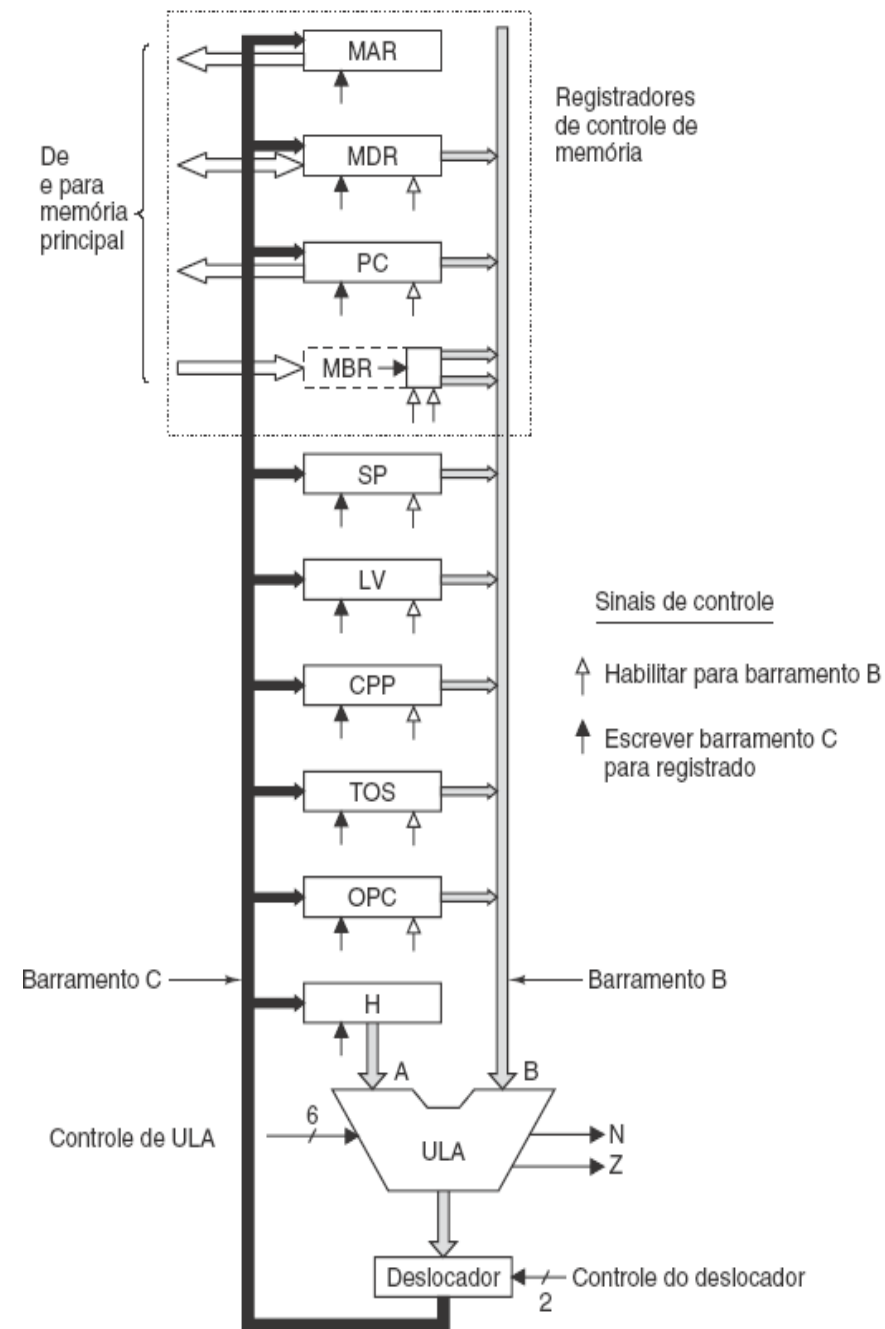
Microinstruções

Para controlar o caminho de dados precisamos de 29 sinais, divididos em 5 grupos:

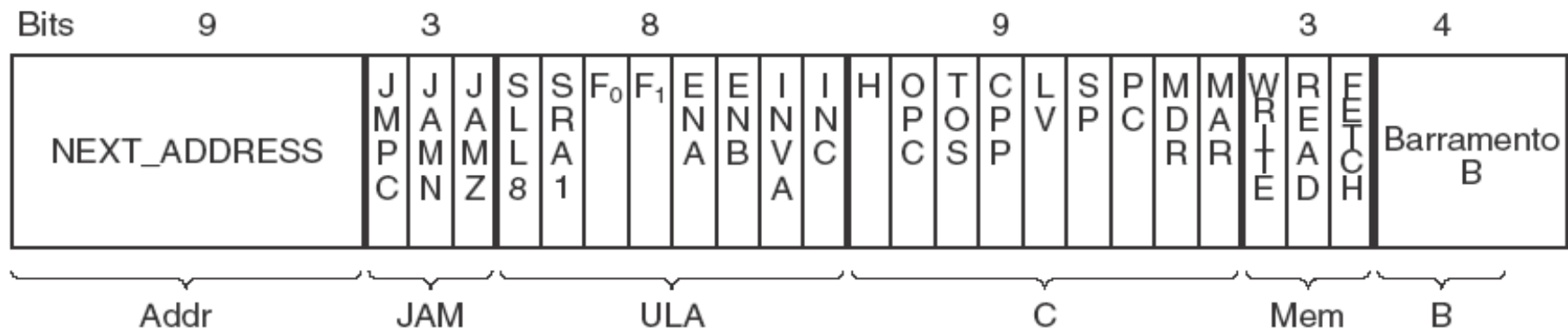
- 9 sinais para escrita de dados do **barramento C**
- 9 sinais para habilitar registradores do **barramento B** para entrada da ALU
- 8 sinais para controlar as funções da **ALU e deslocador**
- 2 sinais (não mostrados) para indicar leitura/escrita na memória via **MAR/MDR**
- 1 sinal (não mostrado) para indicar busca (*fetch*) na memória via **PC/MBR**



- Após algumas análises vemos que no barramento B só poderá ter um único registrador habilitado por vez
- Temos um total de 9 registradores apenas, se tivéssemos 8 registradores poderíamos utilizar um decodificador 3 x 8 para selecionar um dos registradores, mas como são 9 registradores teremos que usar um decodificador 4 x 16, mesmo que tenhamos 7 saídas do decodificador sem utilização
- Com isso termos que o caminho de dados poderá ser controlado com $9 + 8 + 4 + 2 + 1 = 24$ sinais
- Necessitamos de 2 campos adicionais, o campo NEXT_ADDRESS e o campo JAM, totalizando então 36 sinais



- Addr – Endereço da próxima microinstrução
- JAM – Determina como a próxima microinstrução é selecionanda
- ULA – funções da ULA e deslocador
- C – Seleciona qual ou quais registradores serão escritos
- Mem – Funções de memória
- B – Seleciona a fonte do barramento B; é codificado como mostrado

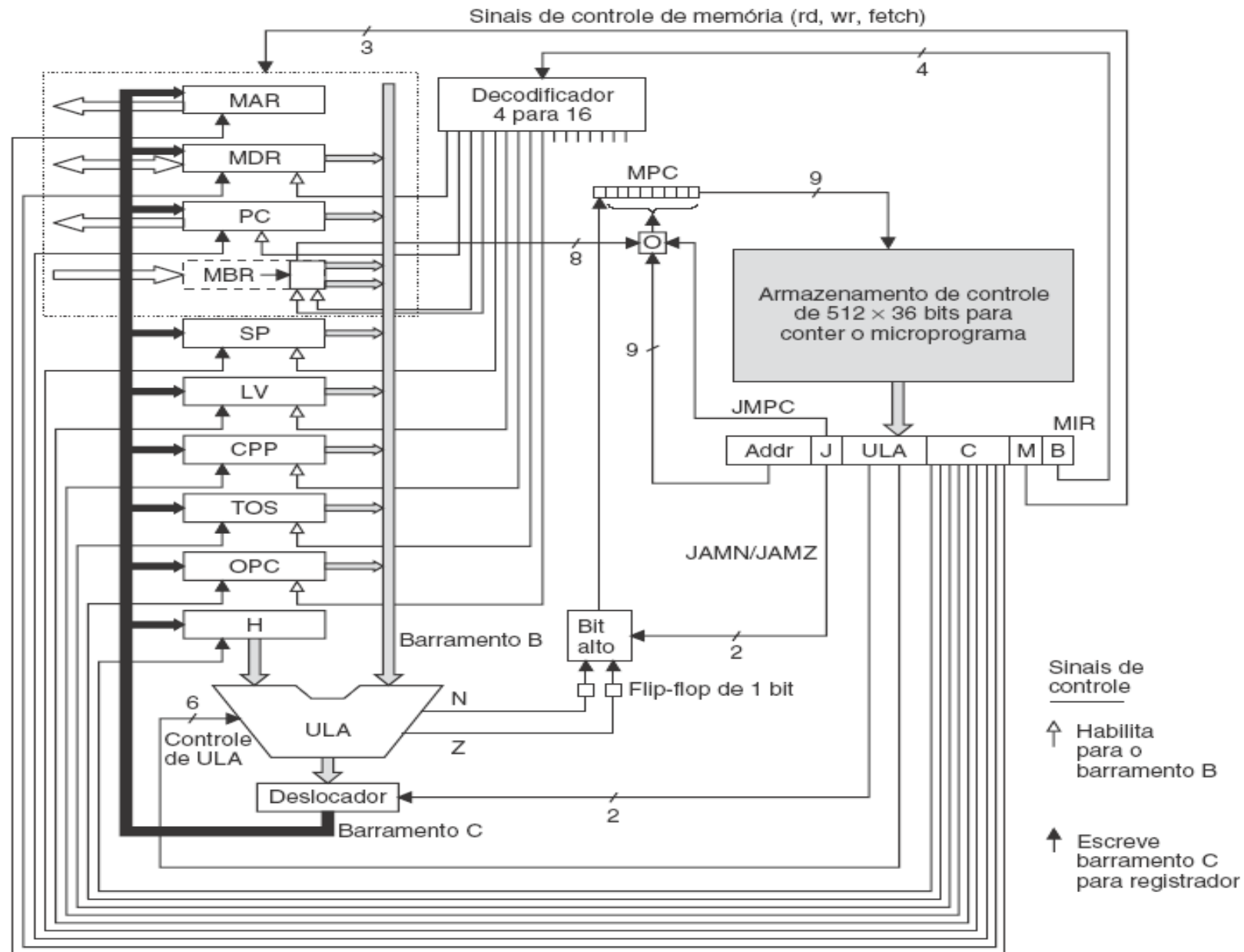


Registradores do barramento B

0 = MDR	5 = LV
1 = PC	6 = CPP
2 = MBR	7 = TOS
3 = MBRU	8 = OPC
4 = SP	9-15 nenhum

Controle de microinstrução: a Mic-1

Diagrama de blocos completo de nosso exemplo de microarquitetura, a MIC-1.

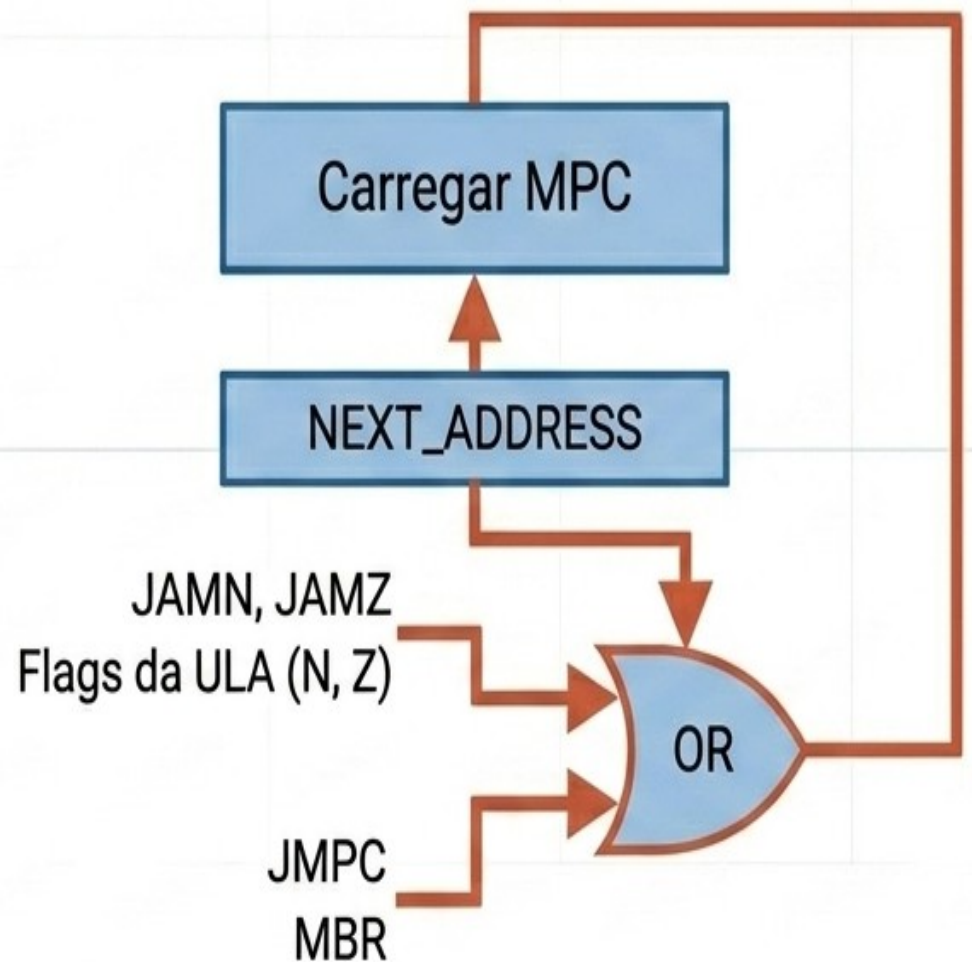


O Sequenciador: Qual é o próximo passo?

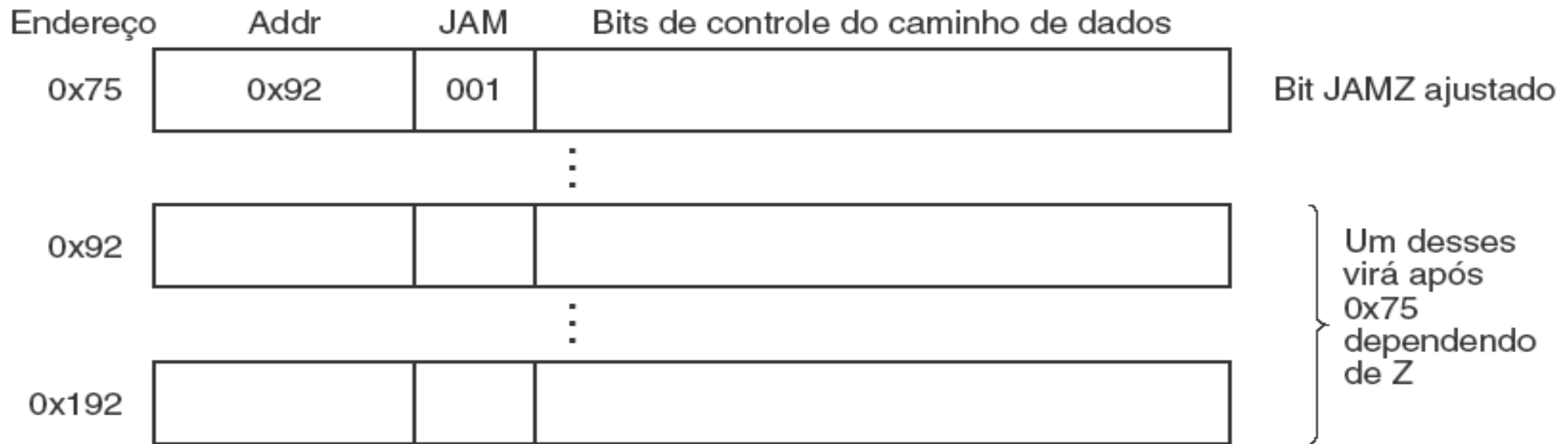
Microprogramas não seguem ordem sequencial automática. Toda instrução declara ativamente sua sucessora via NEXT_ADDRESS.

Os **Bits JAM** realizam o **desvio condicional real no hardware**. Eles aplicam um **OU lógico** sobre o endereço base usando:

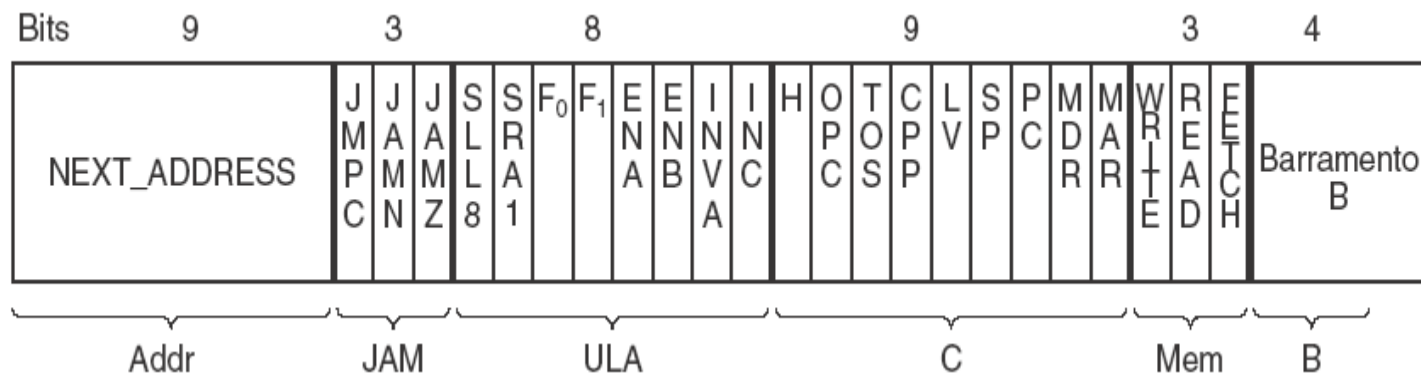
- **Flags da ULA (N, Z)** para a matemática.
- **Conteúdo do MBR** para decodificar opcodes recém-lidos.



Microinstrução com JAMZ



Microinstrução com JAMZ ajustado para 1 tem duas sucessoras potenciais.



Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall